

KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY
(Autonomous Institution)

RENTEASE

RENTAL PROPERTY MANAGEMENT SYSTEM

A MINI PROJECT REPORT

Submitted by

_____

U21CS504 - Web Technologies Laboratory

in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
KPRIET

ARASUR, COIMBATORE - 641 407

NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this project report **"RENTEASE – RENTAL PROPERTY MANAGEMENT SYSTEM"** is the bonafide work of **"_____"** who carried out the project work under my supervision.

Dr. MIRUNA JOE AMALI **HEAD OF THE DEPARTMENT** Professor Computer Science and Engineering KPR Institute of Engineering and Technology Arasur, Coimbatore – 641407	**RAJESHKUMAR S** **SUPERVISOR,** Assistant Professor III Computer Science and Engineering KPR Institute of Engineering and Technology Arasur, Coimbatore – 641407

Submitted for the University Mini project Viva – Voce held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	LIST OF FIGURES	iv
	LIST OF ABBREVIATIONS	v
1	**ABSTRACT**	1
2	**INTRODUCTION**	2
2.1	Background and Need for the Project	2
2.2	Objectives	2
2.3	Scope	3
2.4	Development Plan	3
3	**IMPLEMENTATION**	4
3.1	Tech Stack	4
3.2	Modules	5
3.2.1	Database Module	5
3.2.2	Property Listing and Search Module	6
3.2.3	Application and Payment Module	6
3.2.4	Security and Interface Module	7
3.3	Testing Approach	7
3.4	Workflow of the Application	8
4	**OUTCOME**	11
4.1	Implementation Outcome	11
4.2	Module-wise Outcome with Screenshots	11
4.3	Comparison with Existing Methods	19
4.4	Discussion and Limitations	19
5	**CONCLUSION**	21
5.1	Future Scope	21
	REFERENCES	22

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
3.1	SYSTEM ARCHITECTURE OF RENTEASE	
3.2	ENTITY RELATIONSHIP DIAGRAM OF THE DATABASE	
4.1	HOME PAGE WITH QUICK SEARCH AND FEATURED LISTINGS	
4.2	PROPERTY CATALOGUE WITH THE FILTER PANEL	
4.3	FILTERED CATALOGUE - COIMBATORE, 2 BHK, UNDER RS.20,000	
4.4	LISTING DETAIL WITH GALLERY AND APPLICATION PANEL	
4.5	LOGIN PAGE	
4.6	TENANT DASHBOARD	
4.7	RENT RECORDING FORM	
4.8	MAINTENANCE COMPLAINT FORM AND TIMELINE	
4.9	OWNER DASHBOARD	
4.10	ADD PROPERTY FORM	

LIST OF ABBREVIATIONS

API	APPLICATION PROGRAMMING INTERFACE
CSS	CASCADING STYLE SHEETS
CSRF	CROSS-SITE REQUEST FORGERY
DBMS	DATABASE MANAGEMENT SYSTEM
ER	ENTITY RELATIONSHIP
HTML	HYPERTEXT MARKUP LANGUAGE
HTTP	HYPERTEXT TRANSFER PROTOCOL
IDE	INTEGRATED DEVELOPMENT ENVIRONMENT
JSON	JAVASCRIPT OBJECT NOTATION
MYSQL	MY STRUCTURED QUERY LANGUAGE
OWASP	OPEN WEB APPLICATION SECURITY PROJECT
PDO	PHP DATA OBJECTS
PHP	PHP: HYPERTEXT PREPROCESSOR
REST	REPRESENTATIONAL STATE TRANSFER
SQL	STRUCTURED QUERY LANGUAGE
UI	USER INTERFACE
URL	UNIFORM RESOURCE LOCATOR
XAMPP	CROSS-PLATFORM, APACHE, MARIADB, PHP AND PERL
XSS	CROSS-SITE SCRIPTING

CHAPTER - 1

ABSTRACT

RentEase is a rental property management system built for two audiences that rarely share a tool: property owners who need to keep a portfolio of lettings straight, and tenants who need to find one. The application is written in PHP 8 against MySQL 9, with the front end in semantic HTML5, a single CSS3 stylesheet and a small amount of vanilla JavaScript, and it is delivered as a set of pages that a browser requests rather than a single-page application.

An owner registers, publishes a property with a photograph, rent, deposit, area, furnishing and availability date, and thereafter sees every listing they hold on one dashboard along with the applications it has attracted, the rent collected against it and the maintenance complaints its tenant has raised. A tenant registers, searches the catalogue by keyword, city, bedroom count, property type, furnishing and budget, applies for a property with an occupation and family size, records a monthly rent payment against a transaction reference once approved, and raises maintenance complaints that move from open to in-progress to resolved.

The data layer is six InnoDB tables - users, properties, property_images, applications, payments and complaints - held together by foreign keys and by two unique constraints that carry real business meaning: one application per tenant per property, and one payment per tenant per property per month. Those keys turn duplicate submissions into a database error rather than a duplicated record. Approving an application runs inside a transaction that marks the property occupied and rejects the competing pending applications together, so a property cannot end up with two approved tenants.

Security is applied throughout rather than bolted on. Every query that carries user input is a PDO prepared statement, every POST form carries a CSRF token compared with hash_equals(), every page that needs a role checks it before doing any work, every owner action re-verifies that the record belongs to the signed-in owner, and every value printed back into HTML passes through htmlspecialchars(). Passwords are stored with password_hash() and sessions are regenerated on login.

The completed system was exercised against a live MySQL instance for all six features and fifteen recorded test cases, with screenshots in Chapter 4. RentEase demonstrates server-side scripting, relational modelling with constraints that enforce business rules, session-based authentication, transactional updates and a responsive interface built without a framework.

CHAPTER - 2

INTRODUCTION

2.1 Background and Need for the Project

Renting a house in an Indian city is still largely a brokerage transaction. A tenant rings an agent, the agent rings an owner, and the same flat is quoted at three different rents to three callers. What matters - the real cost, when it is free, whether the deposit is refundable, who fixes the plumbing - lives in the agent's memory and a WhatsApp thread. The owner has no consolidated record of who paid this month or which repair is still open.

The problems with this practice cluster into four kinds: discovery, records, responsibility and trust. They are summarised in Table 2.1. Each one is a bookkeeping failure rather than a hard technical problem, which is what makes it worth solving in software.

Table 2.1 PROBLEMS IN THE EXISTING METHODS OF RENTING

Problem	Description	Impact
Scattered listings	Flats are advertised through individual brokers and forwarded messages.	A tenant cannot compare the market in one place.
No application record	Interest is expressed on the phone and nothing is written down.	Neither side can reconstruct who applied for what, or when.
Informal rent trail	The tenant holds a receipt; the owner holds a notebook.	Monthly reconciliation is manual and disputes are hard to settle.
Untracked maintenance	A repair request is a phone call that leaves no history.	Repairs are forgotten and responsibility is disputed later.
Broker dependency	Contact details sit with the middleman, not with either party.	Both sides pay for an introduction they cannot verify.

Problem statement: how can a single web application let an owner publish and track a portfolio of lettings, and let a tenant search that portfolio, apply, pay and raise complaints, such that every one of those actions leaves a record both parties can read?

2.2 Objectives

- To build a searchable property catalogue carrying an image gallery and a full specification sheet for each listing.
- To let an owner publish a listing, and to let a tenant search by keyword, city, bedroom count, property type, furnishing and maximum rent.
- To record rental applications and give the owner an approve or reject decision that cannot admit two tenants to one property.

- To keep a month-by-month rent payment record that cannot record the same month twice for the same tenant and property.
- To track maintenance complaints through a fixed status path with an owner note.
- To give the owner one dashboard covering listings, applications, rent collected and open complaints.
- To enforce role-based access, CSRF protection and bound SQL parameters on every page that changes state.

2.3 Scope

The delivered system covers owner and tenant registration, the property catalogue with its six filters and four sort orders, the listing detail page with gallery and application form, the application decision, rent recording, complaint tracking and the owner dashboard. It is six database tables, eighteen PHP files, one stylesheet and one script, and it runs either under XAMPP or under PHP's built-in server.

The current version deliberately does not include a payment gateway - rent is recorded and reconciled, not charged - nor an identity or title verification step, an administrator role, e-mail or SMS notification, image resizing or a CDN, full text search, or a GPS proximity search. It also omits listing edit and delete paths, password reset and login rate limiting. These are listed under Future Scope in Chapter 5 rather than presented as complete.

2.4 Development Plan

The work proceeded in six phases, each feeding the next (Table 2.2).

Table 2.2 DEVELOPMENT PLAN OF RENTEASE

S.No	Phase	Activities	Output
1	Requirement analysis	Identified the six features and the two roles, and fixed the scope boundary.	Module list
2	Data design	Designed the six tables, the foreign keys and the unique constraints that encode the business rules.	ER diagram
3	Front end	Built the stylesheet with design tokens, the card grid and the responsive breakpoints.	CSS and layout
4	Back end	Wrote the PDO connection, the session and CSRF helpers, the pages and the transactional actions.	Working application
5	Security	Applied prepared statements, output escaping, role guards and ownership checks.	Hardened application
6	Testing	Exercised every workflow against a live database with valid and invalid input.	Test results

CHAPTER - 3

IMPLEMENTATION

RentEase is a three-tier application in the ordinary sense: a browser renders HTML, a PHP layer decides what that HTML should contain and enforces the rules, and MySQL stores the result. There is no client framework and no build step, so what runs on the server is exactly what is in the source tree, which matters when a report has to explain every line of it.

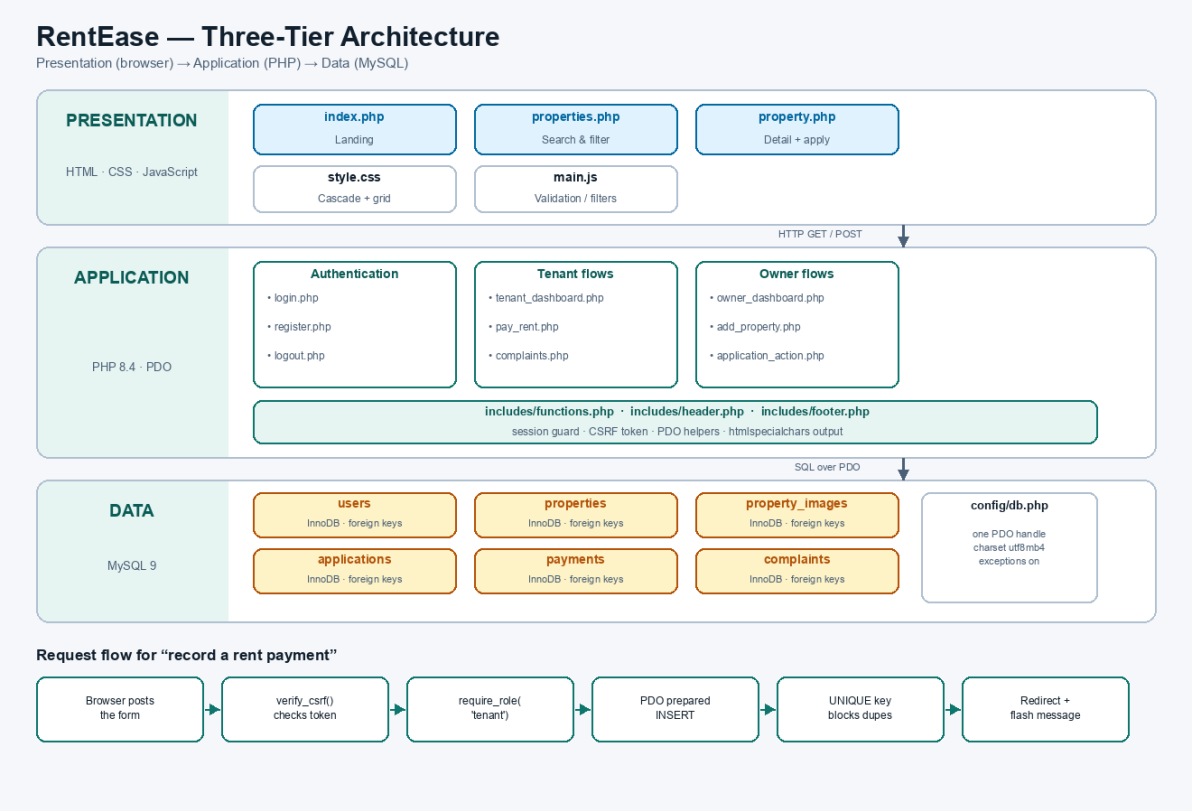


Fig 3.1 SYSTEM ARCHITECTURE OF RENTEASE

3.1 Tech Stack

Table 3.1 TECHNOLOGY STACK

Component	Role in the project
HTML5	Semantic structure: header, nav, section, article, footer, forms, tables.
CSS3	Design tokens as custom properties, grid card layout, flexbox, media queries, hover and transition states.
JavaScript (ES6)	Field validation, live card filtering, gallery switching, deposit preview, flash auto-dismiss.
PHP 8.4	Page rendering, session handling, validation, transactions, file upload.
PDO	One connection, prepared statements everywhere, exceptions enabled.
MySQL 9	Six InnoDB tables, foreign keys, ENUM columns, two unique constraints.
XAMPP / built-in server	Local hosting and phpMyAdmin for importing the schema.
Google Chrome	Testing, responsive checks and screenshot capture.

3.2 Modules

The application is organised as six modules over one shared skeleton. The structure is set out in Table 3.2.

Table 3.2 PROJECT STRUCTURE

File	Purpose
index.php	Landing page: hero, quick search, portfolio statistics, featured listings.
properties.php	Catalogue with keyword, city, bedroom, type, furnishing and budget filters, four sort orders and six-per-page paging.
property.php	Detail page: gallery, specification sheet, owner contact, application form.
register.php / login.php / logout.php	Account creation with role selection, authentication, session teardown.
tenant_dashboard.php	Applications, rent receipts and complaint status for one tenant.
pay_rent.php	Records a monthly payment; amount is read from the property row.
complaints.php	Raise a complaint; shows the status timeline and owner note.
owner_dashboard.php	Portfolio statistics, listing table, application queue, six-month rent chart, complaint queue.
add_property.php	Publishes a listing with a MIME-checked cover image upload.
application_action.php / complaint_action.php / property_action.php	Owner transitions, each re-checking ownership before writing.
config/db.php	Shared PDO handle, utf8mb4, exceptions on.
includes/functions.php	Session bootstrap, require_role, CSRF, e(), money().
assets/css/style.css	All styling: tokens, grid, cards, tables, chips, responsive rules.
assets/js/main.js	All client behaviour.

3.2.1 Database Module

The schema is six tables. users holds owners and tenants in one table distinguished by a role column. properties belongs to an owner and carries the listing facts. property_images holds the gallery. applications joins a tenant to a property with a status. payments records a month of rent. complaints records a maintenance ticket. The relationships and cardinalities are shown in Fig 3.2.

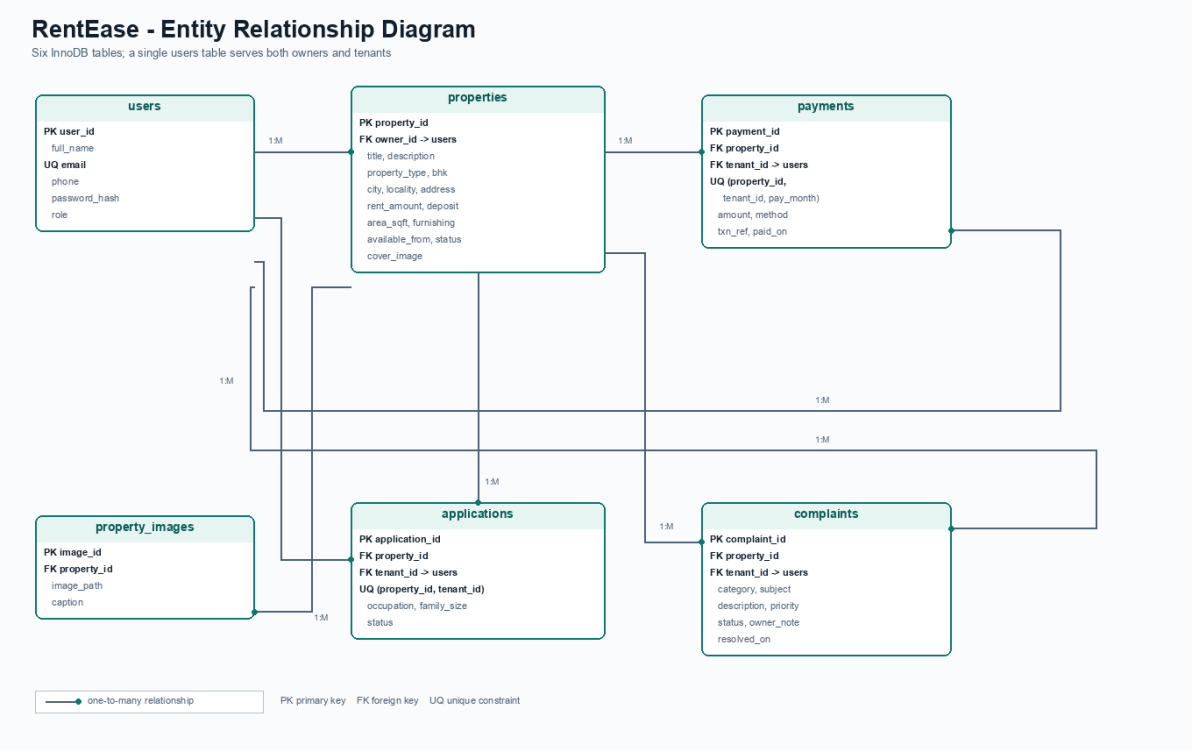


Fig 3.2 ENTITY RELATIONSHIP DIAGRAM OF THE DATABASE

Two constraints are worth calling out because they carry business meaning rather than merely enforcing uniqueness. UNIQUE(property_id, tenant_id) on applications means a tenant applies to a property once; a second attempt is refused by the database. UNIQUE(property_id, tenant_id, pay_month) on payments means a month of rent can be recorded once; a duplicate submission fails at the storage layer even if the page logic is bypassed.

3.2.2 Property Listing and Search Module

The catalogue builds its WHERE clause from only the filters the visitor actually supplied, and binds every value. The ORDER BY is selected from a match() over a fixed set of strings, so no part of the SQL text ever comes from the request. The same query is counted first to compute the page count, then run with LIMIT and OFFSET.

The listing detail page separates concerns by role: a guest is invited to log in, an owner is told that owners cannot apply, a tenant who has already applied sees their status instead of the form, and a tenant who has not sees the application form.

3.2.3 Application and Payment Module

Approving an application is the only multi-row write in the system, and it is the one place where a partial update would be visible to users, so it runs in a transaction: the application

becomes approved, the property becomes occupied, and every other pending application for that property becomes rejected - together or not at all.

Rent recording reads the amount from the properties row rather than from the posted form, so a tenant cannot choose what to pay. The month is validated against a YYYY-MM pattern, the transaction reference must be at least six characters, and the insert relies on the unique key to refuse a duplicate month.

3.2.4 Security and Interface Module

Security controls are listed in Table 3.3. The interface adds a sticky header that collapses to a CSS-only menu under 700 px, status chips that colour-code available, occupied and pending states consistently across every page, and flash messages that confirm an action and then dismiss themselves.

Table 3.3 SECURITY CONTROLS IMPLEMENTED

Control	Implementation
SQL injection	PDO prepared statements with bound parameters; no value is concatenated into SQL.
Cross-site scripting	All user-derived output passes through e(), which applies htmlspecialchars with ENT_QUOTES.
Cross-site request forgery	csrf_field() on every POST; verify_csrf() compares with hash_equals and returns 419 on mismatch.
Broken access control	require_role on every role-specific page; owner actions re-check that the record belongs to the signed-in owner.
Session fixation	session_regenerate_id(true) immediately after a successful login.
Password storage	password_hash() with bcrypt; verification by password_verify().
Unsafe uploads	Cover images checked with mime_content_type against an allow list, capped at 2 MB, stored under a random filename.
Information leakage	PDO exceptions are caught and reported as a generic failure; connection errors never echo credentials.

3.3 Testing Approach

The application was tested locally against a live MySQL 9 instance, first with the seed data and then with deliberately invalid input. Each module was exercised through the browser and its effect confirmed by querying the database directly, so a passing screen did not by itself count as a pass. The scenarios are listed in Table 3.4.

Table 3.4 TEST SCENARIOS AND EXPECTED RESULTS

Test	Input / Action	Expected result
Valid registration	Complete the form with a valid name, email, mobile and a 9-character password.	Row inserted, redirected to the tenant dashboard.
Duplicate email	Register again with an address already in users.	"This email is already registered"; no second row.

Wrong password	Log in with a valid email and an incorrect password.	Invalid credentials message; no session created.
Bad CSRF token	Post the login form with a tampered token value.	HTTP 419; request refused before any query.
Filtered search	Coimbatore, 2 BHK, maximum rent 20,000.	Two of eight listings returned, count updated.
Double application	Apply twice for one property as the same tenant.	Refused by the unique key on (property_id, tenant_id).
Approval	Owner approves one of two pending applications.	Application approved, property occupied, rival rejected.
Cross-owner action	Owner B posts an action for an application of Owner A.	Refused by the ownership check; nothing written.
Duplicate payment	Record rent twice for the same property and month.	Refused by the unique key; one receipt remains.
Complaint resolution	Owner sets a complaint to resolved with a note.	Status and resolved_on set; note visible to the tenant.
Unsafe upload	Upload a .txt file as the cover image.	Rejected by the MIME allow-list.

3.4 Workflow of the Application

The path from opening the site to a settled tenancy is shown in Fig 3.3. The only branch that changes the state of more than one row is the approval decision, which is why it is the only action wrapped in a transaction. The numbered steps follow the figure.

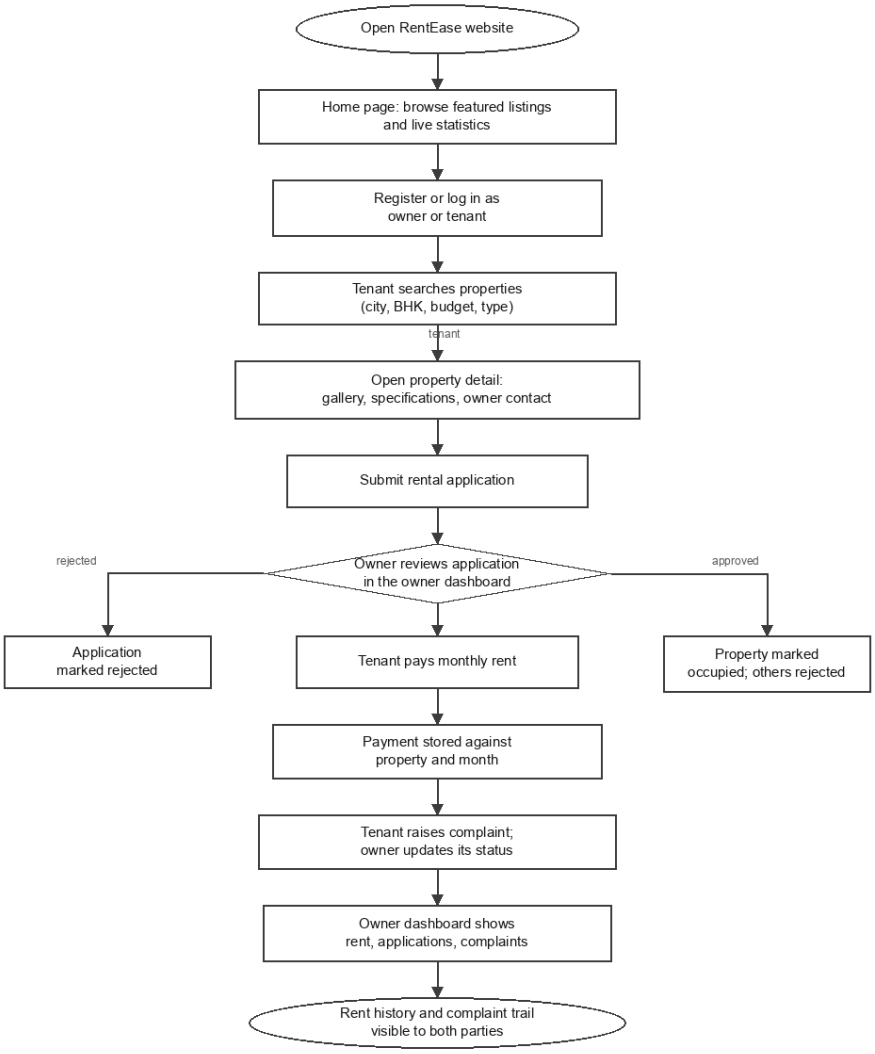


Fig 3.3 WORKFLOW OF THE RENTEASE APPLICATION

1. The visitor opens the site and sees the quick search, the live statistics and six featured listings.
2. A tenant registers or logs in; the role chosen at registration decides which dashboard they reach.
3. The tenant searches the catalogue and opens a listing, where the specification sheet and the owner contact are shown.
4. The tenant submits a rental application with an occupation, family size and message.

5. The owner reviews pending applications on the dashboard and approves one.
6. Approval runs in a transaction: the application is approved, the property becomes occupied and the other pending applications are rejected.
7. The tenant records a rent payment for a chosen month, with the amount taken from the property row.
8. The tenant raises a maintenance complaint; the owner moves it from open through in-progress to resolved with a note.
9. Both parties see the same history on their dashboards, so neither can drift from the record.

CHAPTER - 4

OUTCOME

4.1 Implementation Outcome

RentEase was completed and exercised end to end against a live MySQL 9 instance loaded with the seed data from database/rentease.sql: two owners, three tenants, eight listings across Coimbatore and Chennai, five applications, seven rent receipts and four complaints. Every workflow described in Chapter 3 was driven through the browser and its effect confirmed in the database, not merely observed on screen.

4.2 Module-wise Outcome with Screenshots

The screenshots below were captured from the running application against the seed data. Each is paired with the module it demonstrates.

Home page. A hero section and a five-field quick search sit above a statistics ribbon that reads live from the database - available listings, cities covered, registered tenants and the zero-brokerage claim - followed by six featured listings and a three-step explanation of the workflow.

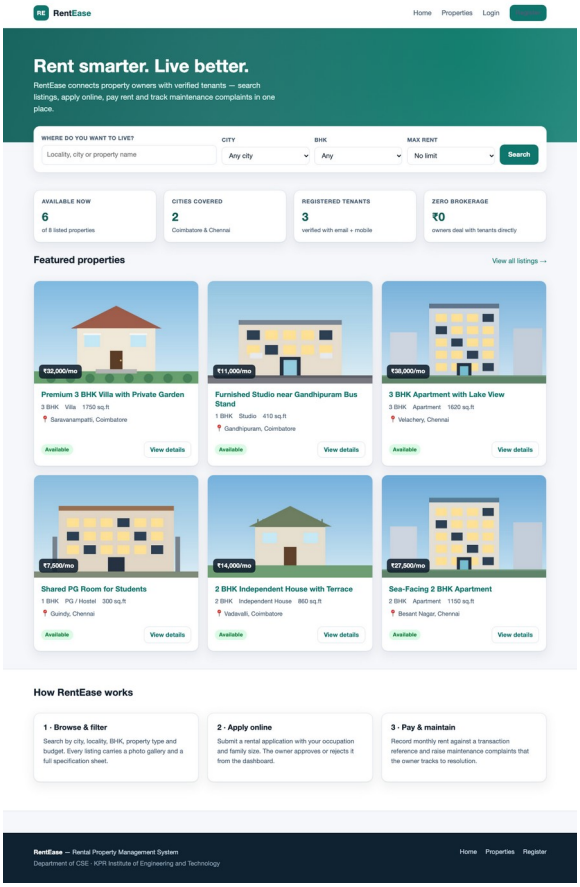


Fig 4.1 HOME PAGE WITH QUICK SEARCH AND FEATURED LISTINGS

Property catalogue. Six filters and four sort orders build a parameterised query; the result count updates and a live text filter narrows the rendered cards without a round trip.

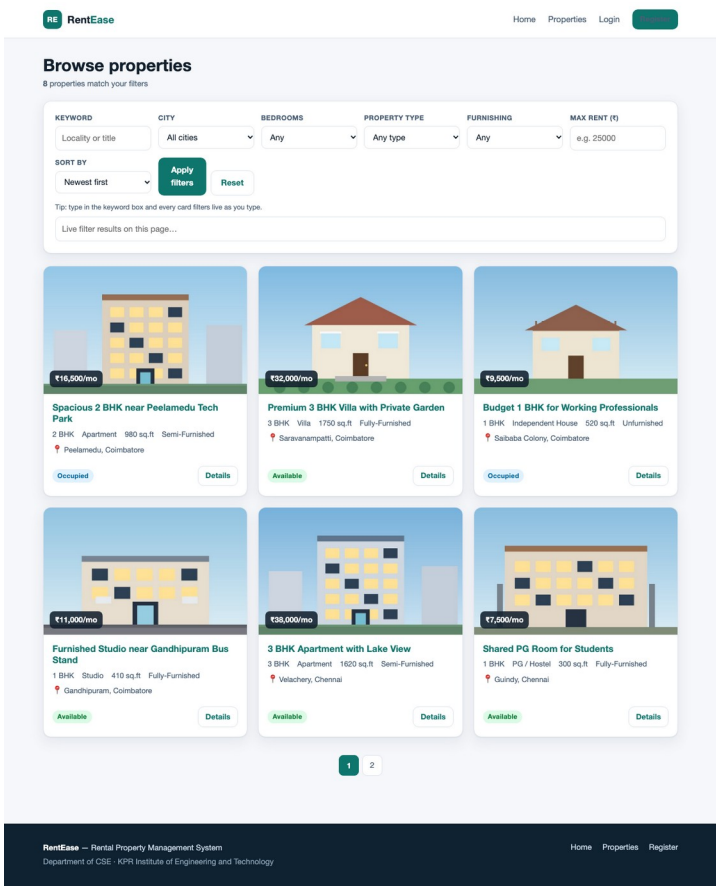


Fig 4.2 PROPERTY CATALOGUE WITH THE FILTER PANEL

Filtered catalogue. Two of the eight seeded listings match the combined filter, and the heading count confirms it.

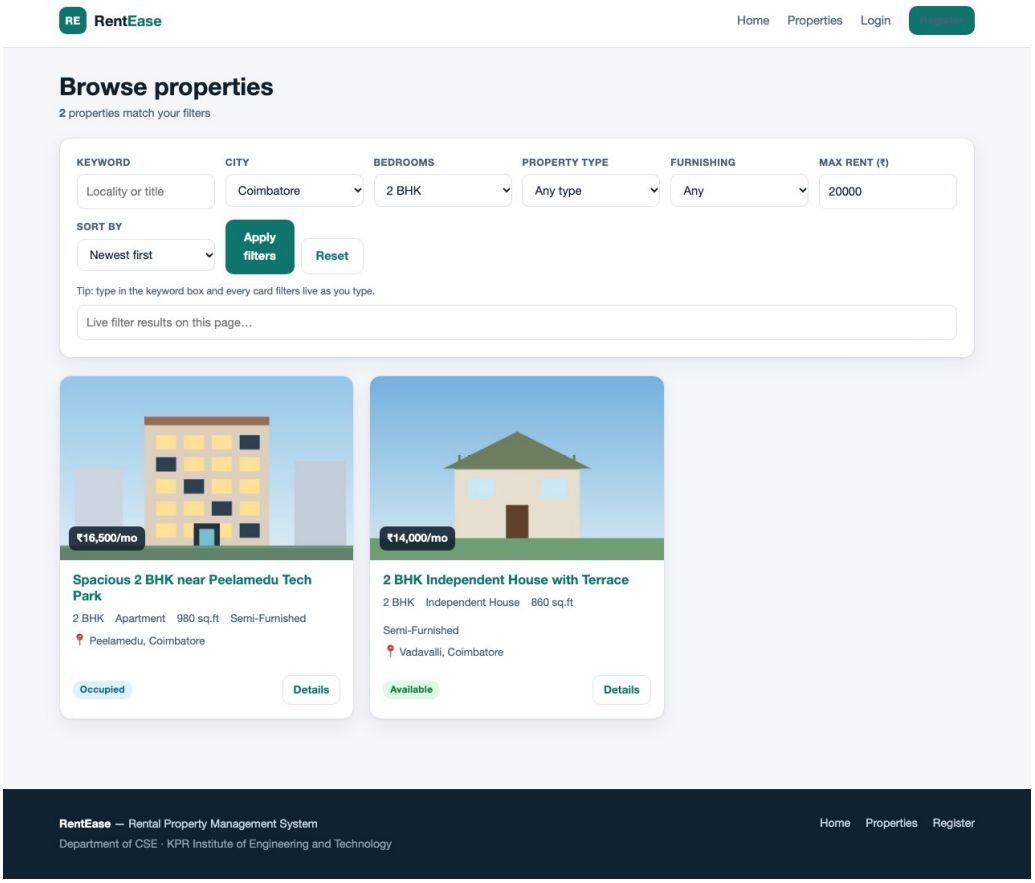


Fig 4.3 FILTER APPLIED: COIMBATORE, 2 BHK, UNDER RS.20,000

Listing detail. The page carries the gallery, the specification sheet and a right-hand panel whose contents depend on who is signed in.

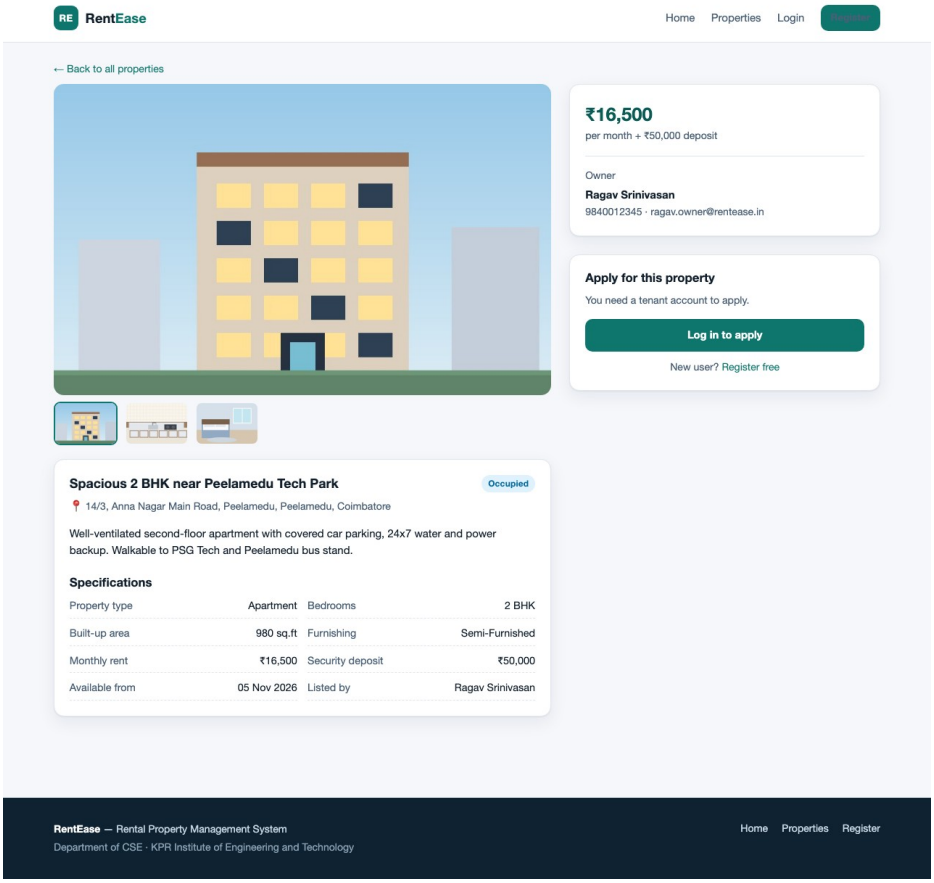


Fig 4.4 LISTING DETAIL WITH GALLERY AND APPLICATION PANEL

Authentication. The demonstration credentials are shown on the page so the marking workflow does not depend on remembering a seed password.

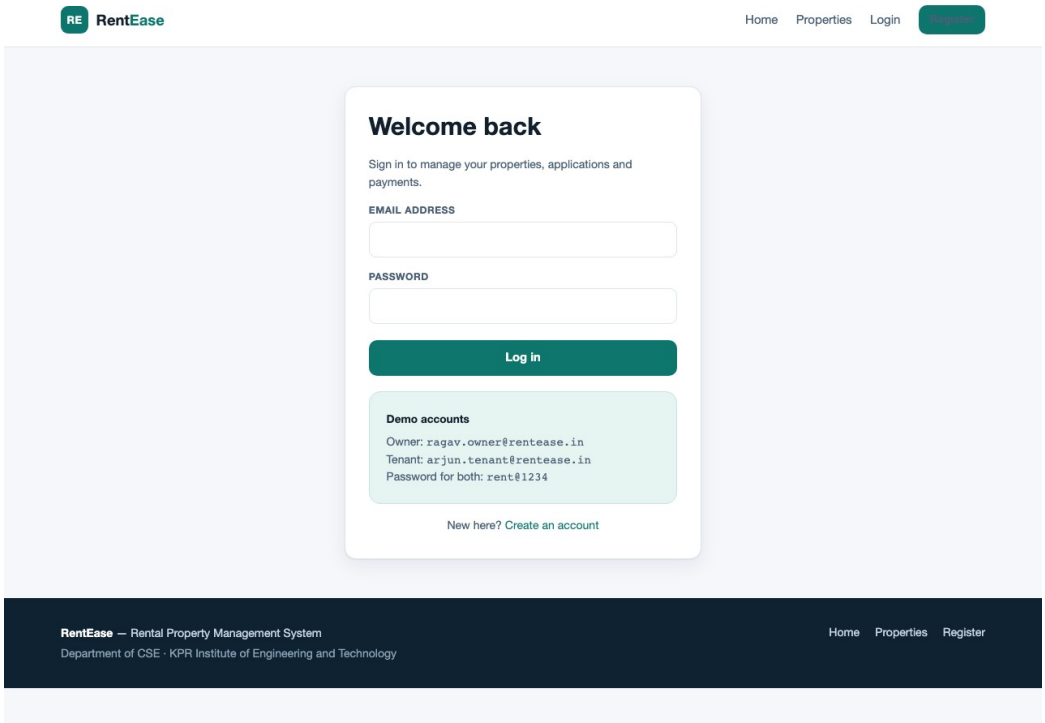


Fig 4.5 LOGIN PAGE

Tenant dashboard. One page carries the applications, the rent receipts and the complaint status, so the tenant never reconciles three screens.

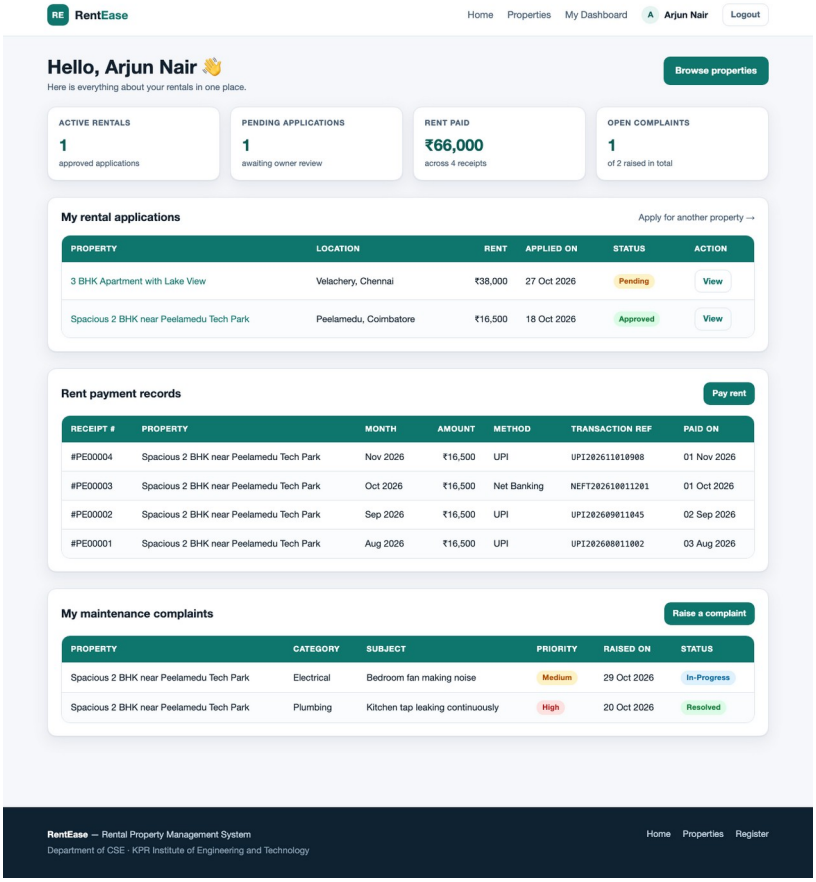


Fig 4.6 TENANT DASHBOARD

Rent recording. The amount is read from the property row rather than the posted form, and the deposit preview is derived on the client.

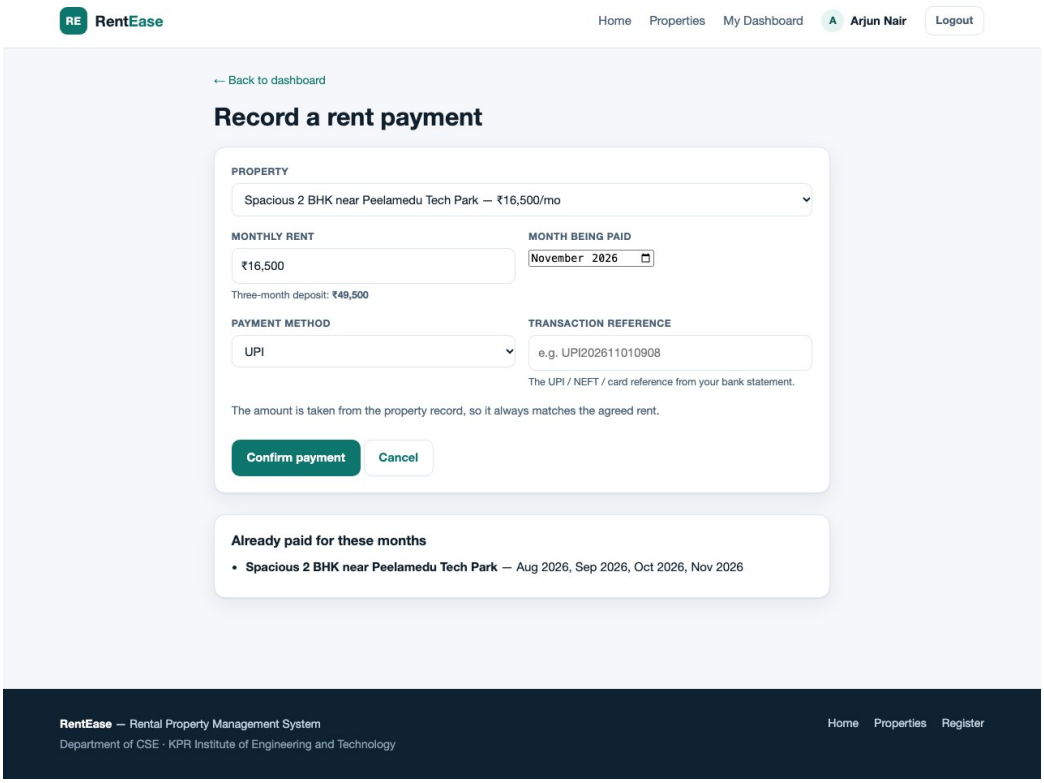


Fig 4.7 RENT RECORDING FORM

Complaints. A complaint is raised against an approved property and shown with its status timeline and the owner's note.

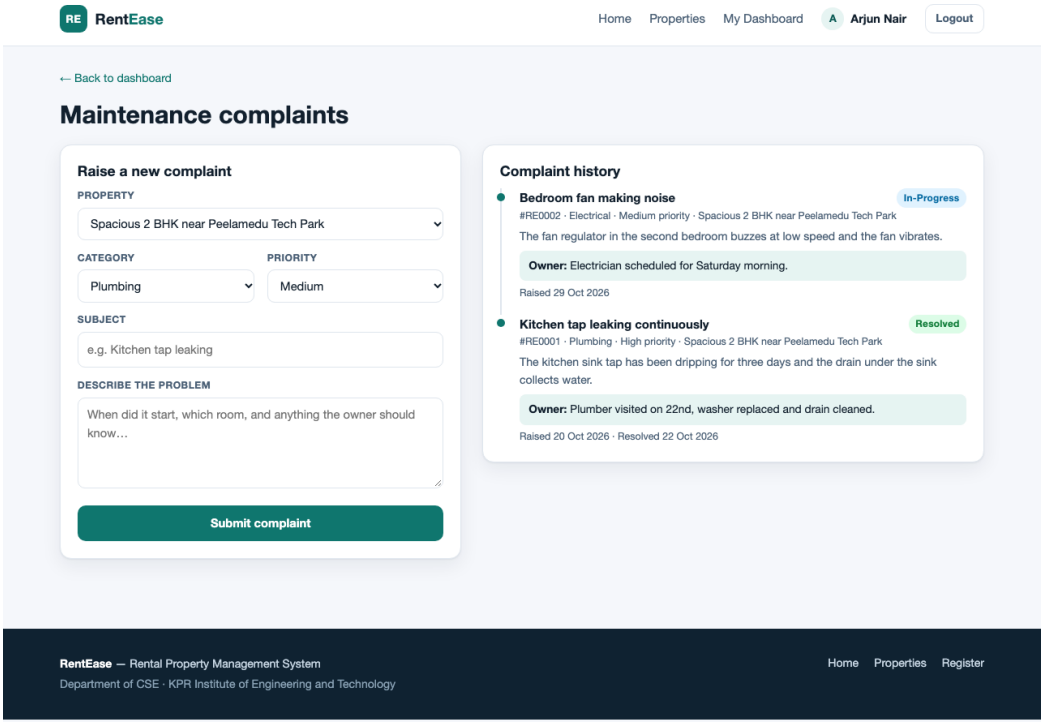


Fig 4.8 MAINTENANCE COMPLAINT FORM AND TIMELINE

Owner dashboard. Listings with pending-application badges, a six-month rent collection chart derived from the payments table, the application queue with approve and reject actions, and the complaint queue with an inline editor.

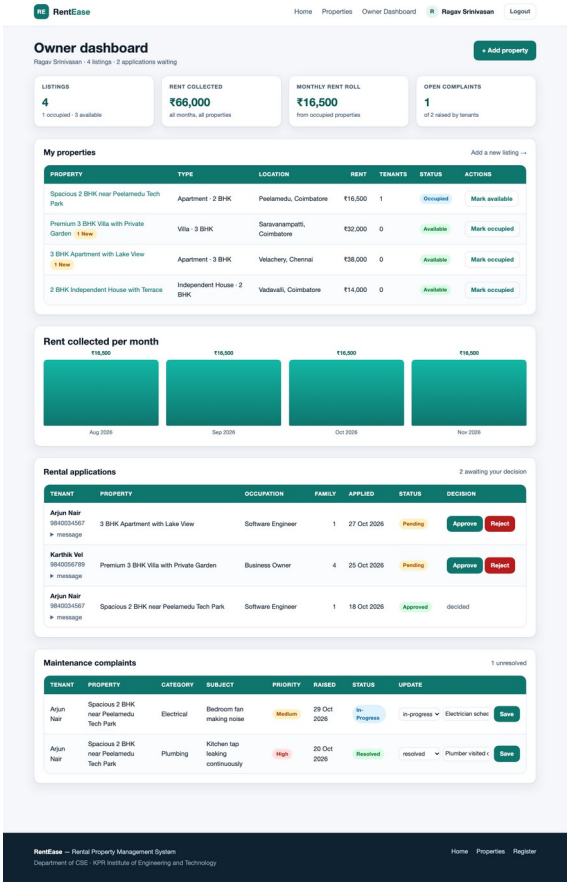


Fig 4.9 OWNER DASHBOARD

Add property. The cover image is validated by MIME type and size before being stored under a random filename.

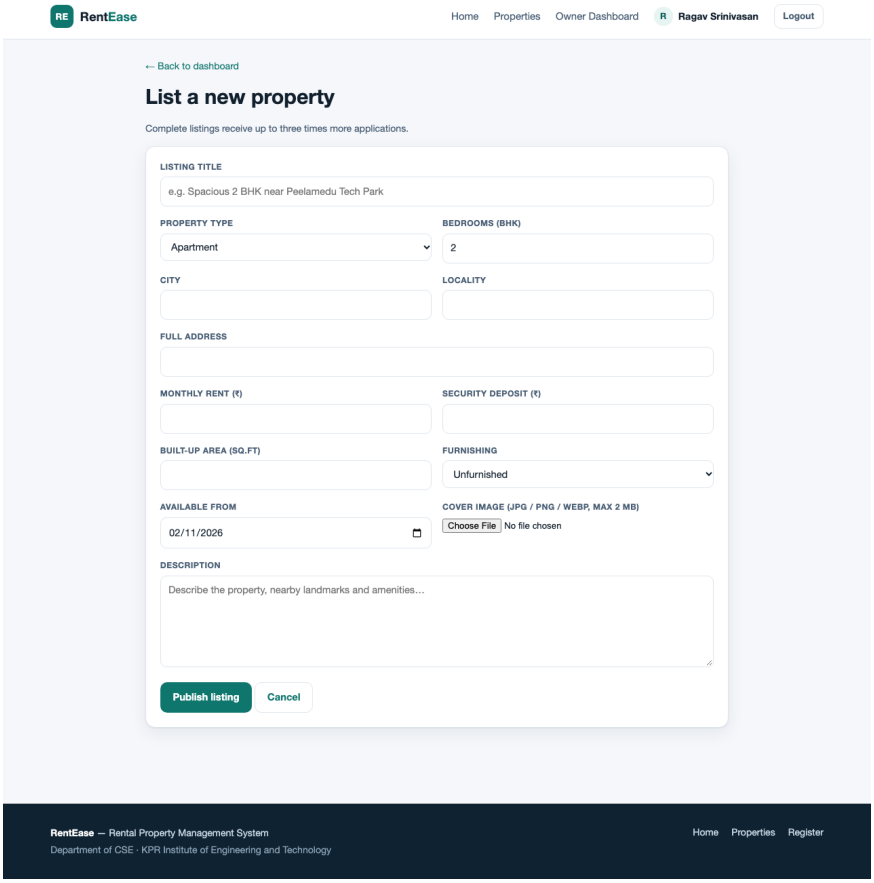


Fig 4.10 ADD PROPERTY FORM

4.3 Comparison with Existing Methods

Table 4.1 sets the delivered system against the broker-based practice described in Chapter 2. The comparison is about where the record of a tenancy lives, not about how many features each side has.

Table 4.1 EXISTING METHODS VERSUS RENTEASE

Aspect	Conventional practice	RentEase
Discovery	A broker quotes flats one at a time.	One catalogue, six filters, four sorts.
Applications	Expressed by phone; nothing written.	A row per tenant per property, with status.
Decision	Applicant told individually.	Transactional approve that occupies the property and rejects the rest.
Rent trail	Tenant holds a receipt, owner a notebook.	One row per month, with method and transaction reference.
Maintenance	Phone call with no history.	Ticket with category, priority, status and owner note.
Oversight	Three separate sources.	One dashboard with statistics and a collection chart.
Access control	Whoever holds the phone number.	Role guard, ownership check, CSRF token, bound parameters.

4.4 Discussion and Limitations

The design decision that shapes everything else is to make the database carry the rules that matter. The two unique keys mean the application does not have to trust the page logic to prevent a duplicate application or a duplicate payment; the transaction around approval means the property and its applications cannot disagree; and reading the rent amount from the property row means the payer cannot choose the price. What is unusual is not any of these mechanisms individually, which are ordinary, but that they live in config/db.php and includes/functions.php so that a new page inherits them rather than re-implementing them.

It is equally worth being explicit about what the system declines to do. It does not negotiate a rent, draft a lease, verify an identity or move money. It records what two parties have already agreed and keeps a history both can read later. That narrowness is why six tables suffice and why the whole workflow can be demonstrated on one machine with eight seeded listings.

The limitations follow from that scope, and several would be disqualifying for a production letting platform. Nothing verifies that an owner holds title to the property they list, and nothing verifies a tenant's identity. Rent is recorded, not collected, so a payment row is an assertion rather than a settled transfer. The 'owner has been notified' message after a complaint is a session flash, not an e-mail or SMS - no mail or messaging code exists in the project. Uploaded images are stored as delivered, with no resizing, no thumbnail derivative and no responsive srcset. There is no edit or delete path for a listing or an application, no password reset, no login rate limiting, and the dashboard queries carry no LIMIT, which is fine at eight listings and would not be at eight thousand. Search is text matching on city and locality rather than distance, so proximity is not modelled at all.

CHAPTER - 5

CONCLUSION

RentEase was built as a rental property management system in PHP 8 and MySQL 9 to replace the informal record-keeping of a residential letting with something both parties can read. An owner publishes a listing, receives applications, approves one, sees the rent arrive month by month and tracks repairs; a tenant searches by the criteria that actually decide a rental, applies, pays and raises complaints. All six features were exercised against a live database and the fifteen recorded test cases passed.

The project demonstrates the practical use of the web technology syllabus: server-side page generation and form handling in PHP, session authentication with hashed passwords, relational design with foreign keys and business-rule constraints, transactional updates, prepared statements and output escaping, semantic HTML5, CSS3 custom properties with grid and flexbox, and client-side validation in vanilla JavaScript.

Its more useful lesson is about modelling. Three of the four rules that actually protect the data - one application per tenant per property, one payment per month, and an approval that occupies the property and rejects its rivals - are enforced by the schema and its transactions rather than by the interface. A system this small can afford to put its guarantees where they cannot be bypassed.

5.1 Future Scope

The following enhancements are outside the current implementation, in rough order of impact.

1. Identity and title verification, with an administrator role to review owners, listings and tenants.
2. A payment gateway so a rent row is a settled transfer, not an assertion.
3. E-mail or SMS notification when a complaint is raised or an application decided, replacing the session-only confirmation.
4. Edit and withdraw paths for listings and applications, with an audit trail.
5. Image processing on upload (resizing, thumbnails, srcset) and object storage.
6. Proximity search on coordinates with distance ranking.
7. Password reset, login throttling and two-factor authentication.
8. Pagination on dashboard queries and indexed full-text search for a larger catalogue.

REFERENCES

1. PHP Documentation. PHP Manual. Available at: https://www.php.net/manual/en/
2. PHP Documentation. PHP Data Objects (PDO). Available at: https://www.php.net/manual/en/book.pdo.php
3. Oracle. MySQL 9 Reference Manual. Available at: https://dev.mysql.com/doc/
4. Apache Friends. XAMPP. Available at: https://www.apachefriends.org/
5. MDN Web Docs. HTML: HyperText Markup Language. Available at: https://developer.mozilla.org/en-US/docs/Web/HTML
6. MDN Web Docs. CSS: Cascading Style Sheets. Available at: https://developer.mozilla.org/en-US/docs/Web/CSS
7. MDN Web Docs. Fetch API. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
8. OWASP Foundation. OWASP Cheat Sheet Series. Available at: https://cheatsheetseries.owasp.org/
9. OWASP Foundation. Cross-Site Request Forgery Prevention Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
10. OWASP Foundation. SQL Injection Prevention Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
11. Python Software Foundation. python-docx Documentation. Available at: https://python-docx.readthedocs.io/
12. The Document Foundation. LibreOffice Writer Guide. Available at: https://documentation.libreoffice.org/